MOBILEWIKI

온디바이스 문서 AI 학습 시리즈

학습 로드맵 / 입문부터 중급까지

안드로이드 문서 AI 개발 학습 계획서

개념을 읽고, 예제로 이해하고, 직접 실습합니다.

01 입문

02 초급

03 중급

이 책에서 다루는 내용

1. 학습 목표와 진행 방식

2. 전체 학습 순서

3. 과목별 개념과 실습

한국어 개념 설명 · 교육 도식 · 예제 · 실습

차례

학습 순서에 따라 읽거나, 항목을 눌러 해당 페이지로 이동하세요.

1. 학습 목표와 진행 방식 . . . 3

2. 전체 학습 순서 . . . 4

3. 과목별 개념과 실습 . . . 5

과목 1. AI와 머신러닝 기본 개념 . . . 5

과목 2. 문서 데이터 처리와 전통적 머신러닝 . . . 7

과목 3. Android 온디바이스 추론 입문 . . . 9

과목 4. 딥러닝 기초와 PyTorch . . . 10

과목 5. 자연어 처리와 사전학습 모델 이해 . . . 12

과목 6. 기존 분류 모델 추가학습 . . . 13

과목 7. 모델 배포와 경량화 . . . 14

과목 8. 생성형 요약과 효율적인 추가학습 입문 . . . 15

4. POC 범위와 검증 기준 . . . 18

5. 학습 기록과 다음 단계 판단 . . . 19

작성 기준: 2026년 10월 18일

책 형태로 읽기: 학습 계획서 PDF

1. 학습 목표와 진행 방식

안드로이드 앱 개발 경험과 Python 기초를 바탕으로, 한국어 문서를 분류하고 요약하는 온디바이스 AI 기능을 개발한다. 쉬운 개념과 작은 예제에서 시작해 기존 모델 추가학습, Android 배포, 경량화까지 중급 수준으로 확장한다.

첫 결과물은 한국어 TXT나 텍스트 PDF를 선택하면 인터넷 없이 문서를 분류하고 핵심 문장을 보여 주는 Android 앱이다. 이후 생성형 요약을 실험하고, 실제 기기에서 적용할 가치가 있는지 판단한다.

학습 환경과 기본 범위

항목	기본 방향
학습자	Android 개발 경험과 Python 기초가 있는 AI 비전공자
학습·추가학습	일반 PC·Mac에서 기초 실습, GPU가 필요한 실습은 Colab 활용
앱 구현	Kotlin, CPU 추론부터 시작
테스트	Android 에뮬레이터와 중급형·구형 실기기
첫 입력	한국어 TXT와 텍스트를 복사할 수 있는 단순한 PDF
첫 요약	원문의 핵심 문장을 선택하는 추출 요약
후속 확장	기존 모델 추가학습, 양자화, 생성형 요약, LoRA 입문

온디바이스 AI는 모델의 실행 위치가 사용자 기기인 방식이다. PC에서 학습한 모델을 휴대폰에서 실행해도 온디바이스 AI다. 휴대폰 자체에서 모델을 학습시키는 작업은 기본 범위에 포함하지 않는다.

과목별 학습 방법

각 과목은 쉬운 개념 → 작은 예제 → 직접 실습 → 이해 확인 → POC 연결 순서로 진행한다. 기간이나 주차 대신 완료 기준을 충족했는지로 다음 단계 진입을 판단한다.

- 입문: 개념을 자신의 말로 설명하고 예제의 입력과 결과를 이해한다.
- 초급: 예제를 실행하고 입력이나 설정을 바꿔 결과 차이를 설명한다.
- 중급: 모델을 추가학습 · 배포하고 품질과 실행 비용을 비교해 선택한다.

2. 전체 학습 순서

순서	학습 분야	난이도	연결 결과물
1	AI와 머신러닝 기본 개념	입문	문서 분석 기능의 입력과 출력 정의
2	문서 데이터 처리와 전통적 머신러닝	입문 → 초급	POC A: PC 문서 분석기
3	Android 온디바이스 추론 입문	초급	POC B: 첫 오프라인 앱
4	딥러닝 기초와 PyTorch	초급	작은 신경망 학습 실습
5	자연어 처리와 사전학습 모델 이해	초급	한국어 모델과 tokenizer 실행
6	기존 분류 모델 추가학습	중급	내 문서에 맞춘 분류 모델
7	모델 배포와 경량화	중급	POC C: 추가학습·양자화 모델 적용 앱
8	생성형 요약과 효율적인 추가학습 입문	중급	POC D: 생성형 요약 비교 실험



그림 1. 분야별 학습 순서와 POC 연결. 기간보다 완료 기준을 중심으로 진행합니다.

1~3번으로 작은 앱을 먼저 완성한다. 이후 4~6번에서 모델의 원리와 추가학습을 배우고, 7번에서 기기에 맞게 최적화한다. 8번의 PC 생성형 요약 실험은 5번 이후 시작해 6~7번과 병행할 수 있다.

3. 과목별 개념과 실습

과목 1. AI와 머신러닝 기본 개념

학습 자료: 과목 1 AI와 머신러닝 기본 개념 · 학습서 PDF

목표: 모델이 어떤 역할을 하는지 이해하고, 만들 기능을 구체적인 문제로 표현한다.

쉬운 개념

- **모델**: 입력을 받아 결과를 내는, 학습된 계산 규칙이다.
- **파라미터·가중치**: 모델 안에서 학습으로 조정되는 숫자다.
- **학습**: 데이터에서 패턴을 배우며 모델의 숫자를 조정하는 과정이다. 이 과정의 문서 분류에서는 정답 라벨을 사용한다.

- 추론: 학습한 모델로 새로운 입력의 결과를 구하는 과정이다.
- 라벨: 예제에 붙이는 정답이다.

Android 개발에 연결하면 모델 파일은 앱이 읽는 데이터이고, 추론 코드는 그 데이터를 사용해 결과를 계산하는 코드다. 모델 파일만 앱에 넣는 것으로 기능이 완성되지는 않으며 입력 처리와 실행 코드가 함께 필요하다.

작은 예제

문서 내용 예시	정답 라벨
신형 스마트폰의 기능을 소개한다	IT
기업의 매출과 실적을 설명한다	경제
축구 경기의 결과를 전한다	스포츠

사람이 붙인 정답으로 모델을 학습한 뒤, 처음 보는 문서의 주제를 예측하는 흐름을 생각해 본다. IT 기업의 실적처럼 두 주제에 걸치는 문서는 먼저 분류 기준을 정해야 한다.

분류와 요약도 구분한다. 분류는 정해진 종류를 선택하고, 추출 요약은 원문 문장을 선택하며, 생성형 요약은 새 문장을 작성한다.

직접 실습

1. 짧은 한국어 문서 30개를 모아 IT · 경제 · 스포츠로 직접 분류한다.
2. 판단하기 어려운 문서를 따로 모으고, 어느 내용을 기준으로 분류할지 적는다.
3. 원하는 앱의 입력, 출력, 실패 상황을 각각 한 문장으로 정의한다.

이해 확인과 완료 기준

☐ 학습과 추론의 차이를 설명할 수 있다.

☐ 분류 · 추출 요약 · 생성형 요약의 차이를 예로 들 수 있다.

☐ 모델을 만들기 전에 정답 기준을 정해야 하는 이유를 설명할 수 있다.

과목 2. 문서 데이터 처리와 전통적 머신러닝

목표: 텍스트를 숫자로 표현하고 작은 분류기를 직접 학습한다. 이 모델을 이후 모델의 비교 기준인 기준선으로 사용한다.

쉬운 개념

- **전처리**: 모델에 넣기 전에 입력을 일정한 형태로 정리하는 작업이다.
- **Bag of Words**: 문서에 어떤 단어가 얼마나 등장하는지 숫자로 나타내는 방법이다.
- **TF-IDF**: 한 문서에서 눈에 띄면서 전체 문서에 너무 흔하지 않은 단어에 더 큰 값을 주는 표현 방법이다.
- **로지스틱 회귀**: 입력 숫자를 이용해 문서 종류를 예측하는 분류 방법이다.
- **추출 요약**: 문장별 중요도를 계산해 원문에서 핵심 문장을 고르는 방법이다.

TXT는 파일에서 텍스트를 읽고, 텍스트 PDF는 문서에서 글자를 추출한다. 스캔 PDF는 이미지 속 글자를 읽는 OCR이 추가로 필요하므로 첫 실습에서는 제외한다.

작은 예제

스포츠 문서에는 '경기', '선수', '득점'이 자주 등장할 수 있다. 문서를 이러한 단어들의 숫자 표현으로 바꾸고, 그 숫자로 주제를 분류한다. 단어 하나만으로 정답이 결정되는 것은 아니라는 점도 오답에서 확인한다.

요약은 문장마다 TF-IDF를 이용한 중요도 점수를 계산하고, 중요한 문장 최대 3개를 골라 원문 순서대로 표시하는 방식으로 시작한다. 이 요약에는 별도 신경망 학습이 필요하지 않다.

데이터와 평가 개념

개념	쉬운 설명
학습 데이터	모델이 공부하는 자료
검증 데이터	학습 설정과 모델을 선택할 때 확인하는 자료
테스트 데이터	선택이 끝난 뒤 마지막으로 평가하는 자료
과적합	공부한 예제는 잘 맞히지만 새로운 예제는 잘 맞히지 못하는 상태
정밀도 precision	특정 종류라고 예측한 결과 중 실제로 맞은 비율
재현율 recall	실제 해당 종류인 문서 중 모델이 찾아낸 비율
macro-F1	종류별 정밀도와 재현율을 함께 반영한 F1을 종류마다 고르게 평균한 값
혼동행렬	어떤 종류를 다른 종류로 잘못 판단했는지 보여 주는 표

예를 들어 스포츠라고 예측한 문서 10개 중 8개가 맞으면 정밀도는 80%다. 실제 스포츠 문서가 12개이고 그중 8개를 찾았다면 재현율은 약 66.7%다.

직접 실습

1. Python에서 TXT를 읽고 텍스트와 라벨을 저장한다. 단순한 PDF는 그다음에 연결한다.
2. 문단 경계를 유지하며 불필요한 공백과 반복 문구를 정리한다.
3. 문서 수를 약 300개로 늘리고 학습 · 검증 · 테스트로 분리한다.
4. scikit-learn으로 TF-IDF와 로지스틱 회귀 분류기를 학습한다.
5. 핵심 문장 선택 기능을 추가하고, 틀린 분류와 좋지 않은 요약을 직접 읽는다.

어휘와 IDF 같은 전처리 기준도 학습 데이터에서만 구한다. 테스트 자료를 미리 사용하면 새로운 문서에 대한 성능을 과대평가할 수 있다.

이해 확인과 완료 기준

- [ ] 텍스트를 숫자로 바꾸는 이유를 설명할 수 있다.
- [ ] 학습한 모델을 저장하고 다시 읽어 같은 문서를 분석할 수 있다.
- [ ] 오답을 보고 데이터 · 분류 기준 · 모델 중 어느 부분이 원인인지 가설을 세울 수 있다.

산출물: POC A, 학습 · 평가 결과, 오답 예시.

자료: scikit-learn 텍스트 특징 추출, 데이터 누출 방지.

과목 3. Android 온디바이스 추론 입문

목표: PC에서 만든 기준선을 앱에 연결해 전체 사용자 흐름을 완성한다.

쉬운 개념

- **모델 로딩**: 저장한 모델 정보를 메모리로 읽는 작업이다.
- **추론 실행**: 입력에 학습된 계산을 적용하는 작업이다.
- **입력 일치**: PC와 앱이 문서를 같은 규칙으로 처리하는 것이다.
- **실행 비용**: 결과를 얻는 데 필요한 시간, 메모리, 저장공간 등이다.

첫 앱은 어휘 · IDF · 분류 가중치를 파일로 읽고 Kotlin에서 같은 계산을 수행한다. 작은 전통적 모델로 앱 전체 흐름을 익힌 뒤, 신경망 실행 도구를 연결한다.

작은 예제

PC와 앱에 같은 문서를 넣었는데 결과가 다르다면, 모델 문제라고 바로 판단하지 않는다. 공백 처리, 단어 분리, 어휘 번호, 숫자 정규화가 같은지부터 확인한다.

직접 실습

1. 문서를 선택하고 텍스트를 추출한다.
2. 백그라운드에서 분류와 추출 요약을 실행한다.
3. 화면에 원문, 분류 결과, 핵심 문장을 표시한다.
4. 준비 중 · 분석 중 · 완료 · 오류 상태와 취소 동작을 처리한다.

5. 비행기 모드에서 실행하고, 모델 로딩과 분석 시간을 나누어 측정한다.

실기기의 Android 버전, 메모리, 32 · 64비트 실행 환경을 기록한다. 에뮬레이터는 앱 연결과 결과 확인에 사용하며 실기기의 속도 · 발열 · 배터리를 대신 평가하는 도구로 사용하지 않는다.

이해 확인과 완료 기준

☐ 같은 문서에 대해 PC와 앱의 결과가 일치한다.

☐ 오프라인에서도 전체 흐름이 동작한다.

☐ 연속 실행과 취소 후 재실행이 가능하다.

☐ 처리 시간이 길어도 UI가 멈추지 않는다.

산출물: POC B와 첫 실기기 측정 기록.

과목 4. 딥러닝 기초와 PyTorch

목표: 기존 모델의 추가학습 코드를 읽을 수 있도록 신경망 학습의 기본 흐름을 이해한다.

쉬운 개념

개념	쉬운 설명
Tensor	숫자를 배열로 담는 자료 구조
Shape	배열의 각 방향에 숫자가 몇 개 있는지 나타낸 크기
순전파	입력에서 예측 결과를 계산하는 과정
Loss	예측이 정답에서 얼마나 벗어났는지 나타내는 값
Gradient	오차를 줄이기 위해 숫자를 어느 방향으로 바꿀지 알려 주는 정보
역전파	가중치별 gradient를 계산하는 과정
Optimizer	gradient를 사용해 가중치를 갱신하는 방법
Batch	한 번의 계산에 함께 사용하는 예제 묶음
Epoch	학습 데이터를 한 차례 사용하는 단위
Learning rate	가중치를 한 번에 얼마나 조정할지 정하는 값

작은 예제

과목 2의 문서 숫자 표현을 작은 신경망에 넣어 분류한다. 반복 학습으로 학습 오차는 줄지만 검증 성능은 나빠지는 상황을 살펴본다. 이를 통해 오래 학습하는 것이 항상 좋은 것은 아니라는 점을 배운다.

직접 실습

1. PyTorch에서 작은 tensor를 만들고 shape를 확인한다.
2. 작은 분류 신경망으로 예측 → 오차 계산 → 가중치 갱신을 반복한다.
3. 학습률과 반복 횟수를 바꾸고 학습 · 검증 곡선을 비교한다.
4. 학습 · 평가 모드를 구분하고 모델을 저장 · 재로딩한다.

수학은 필요한 순간에 병행한다. 벡터 · 행렬 · 내적은 모델 계산을 이해할 때, 미분 · 기울기는 학습 원리를 이해할 때 다룬다. 미분 공식 증명이나 역전파 수식 유도를 완료 조건으로 삼지 않는다.

이해 확인과 완료 기준

☐ 학습 코드에서 입력 · 예측 · loss · 가중치 갱신 부분을 찾을 수 있다.

☐ 학습 결과가 나빠졌을 때 무엇을 확인할지 설명할 수 있다.

☐ 모델을 저장하고 재로딩해 추론할 수 있다.

자료: PyTorch 한국어 입문.

과목 5. 자연어 처리와 사전학습 모델 이해

목표: 한국어를 이미 학습한 모델을 불러오고 입력과 출력을 이해한다.

쉬운 개념

- Token: 모델이 텍스트를 처리하는 조각이다. 한 글자나 한 단어와 항상 같지는 않다.
- Tokenizer: 텍스트를 토큰과 숫자 번호로 바꾸는 도구다.
- Embedding: 토큰을 계산 가능한 숫자 벡터로 표현하는 방법이다.
- Attention: 입력의 여러 부분 사이 관계를 반영하는 계산이다.
- Transformer: attention 등을 사용하는 모델 구조다.
- 사전학습: 많은 자료로 언어의 패턴 등을 미리 학습하는 과정이다.

분류에 사용할 encoder 계열 모델과 새 문장을 작성하는 생성형 모델의 역할을 구분한다. Transformer 전체를 직접 구현하는 대신 기존 모델을 실행하며 구조를 이해한다.

작은 예제

‘온디바이스 AI로 문서를 요약한다’ 를 tokenizer에 넣어 실제 토큰과 번호를 확인한다. 같은 문장도 tokenizer에 따라 나뉘는 방식이 다를 수 있으므로 모델에 맞는 tokenizer를 사용한다.

긴 문서는 앞부분만 잘라 넣어 본다. 뒤쪽에만 핵심 내용이 있는 문서를 비교해 입력 길이 제한이 결과에 미치는 영향을 확인한다.

직접 실습

1. Hugging Face에서 한국어 소형 모델과 tokenizer를 불러온다.
2. 토큰 번호와 padding · attention mask를 확인한다. Padding은 길이를 맞추는 채움이고, attention mask는 실제 입력과 채움을 구분하는 정보다.
3. 짧은 문장과 긴 문서의 토큰 수, 처리 시간, 출력 형태를 비교한다.
4. 문단을 기준으로 긴 문서를 나누는 실험을 한다.

이해 확인과 완료 기준

- [ ] 글자 수와 토큰 수가 다른 이유를 설명할 수 있다.
- [ ] 모델과 tokenizer를 함께 저장해야 하는 이유를 이해한다.
- [ ] 긴 문서를 조용히 잘라 처리할 때 생기는 문제를 설명할 수 있다.

과목 6. 기존 분류 모델 추가학습

목표: 사전학습 모델을 내 문서의 분류 기준에 맞게 조정하고 기준선과 비교한다.

쉬운 개념

- **전이학습**: 이미 배운 모델을 새로운 작업에 활용하는 방법이다.
- **Fine-tuning**: 내 데이터로 기존 모델을 추가 조정하는 과정이다.
- **분류 head**: 모델의 표현을 받아 원하는 종류의 점수를 계산하는 출력 부분이다.
- **모델 선택**: 검증 결과를 보고 어떤 설정과 모델을 사용할지 정하는 작업이다.

작은 예제

한국어를 미리 학습한 모델에 IT · 경제 · 스포츠를 구분하는 출력 부분을 붙이고 문서 예제로 추가학습한다. 기존 모델이 내 분류 기준까지 이미 알고 있다고 가정하지 않는다.

첫 실습 모델은 KoELECTRA-Small의 monologg/koelectra-small-v3-discriminator로 한다. 이 모델은 분류 실습에 사용하며 요약 문장을 생성하는 용도로 사용하지 않는다.

직접 실습

1. 본격적인 학습 전에 샘플 분류 모델을 ONNX로 변환하고 Android CPU에서 한 번 실행해 배포 경로를 확인한다.
2. 같은 문서 분할을 사용해 소형 모델을 추가학습한다.
3. 검증 데이터로 학습 설정을 고르고, 고정한 모델을 테스트 데이터에서 평가한다.
4. TF-IDF 분류기와 macro-F1, 혼동행렬, 실행 비용을 비교한다.
5. 학습한 모델, tokenizer, 라벨 순서와 전처리 설정을 함께 저장한다.

이해 확인과 완료 기준

☐ 추가학습 전후 무엇이 달라졌는지 설명할 수 있다.

☐ 특정 분류의 성능만 나쁜 경우를 찾아 분석할 수 있다.

☐ 더 복잡한 모델을 채택할지 기준선 모델을 유지할지 근거로 판단할 수 있다.

자료: Hugging Face 문서 분류, 모델 카드 읽기.

과목 7. 모델 배포와 경량화

목표: 추가학습 모델을 앱에서 실행하고 품질 · 크기 · 속도 · 메모리의 균형을 판단한다.

쉬운 개념

- **ONNX**: 모델의 계산을 담는 배포 형식 중 하나다.
- **Runtime**: 모델 파일을 읽고 계산을 실행하는 도구다.
- **양자화**: 모델의 숫자를 더 적은 비트로 표현하는 방법이다.
- **FP32·FP16·INT8**: 숫자를 표현하는 형식이다. 각각의 지원과 효과는 모델 · 실행 도구 · 기기에 따라 다르다.
- **학습 후 양자화**: 학습을 끝낸 모델에 양자화를 적용하는 방법이다.

모델 파일 크기는 저장공간이고, 실행 메모리는 계산 중 필요한 공간이다. 둘은 같지 않으며, 작은 파일이 반드시 더 빠르게 실행되는 것도 아니다.

작은 예제

같은 문서에 원본 모델과 INT8 모델을 적용한다. 분류 결과가 달라진 문서를 살펴보고, 파일 크기와 실기기 분석 시간을 함께 기록한다.

직접 실습

1. ONNX Runtime CPU로 Android에서 신경망 추론을 실행한다.
2. Python과 Android에서 동일 문서의 토큰 번호와 예측 결과를 비교한다.
3. 원본 모델에 INT8 양자화를 적용한다.
4. 같은 입력 길이와 기기 조건에서 원본 · 양자화 모델을 비교한다.
5. 긴 문서는 문단별로 처리하고 분류 점수를 모으는 방식을 실험한다. 앞부분만 처리한 경우와 비교한다.

양자화 대상 연산과 실행 환경의 지원을 먼저 확인한다. CPU 결과를 확보한 뒤 GPU · NPU 가속을 검토한다. 구형 기기에서는 모델 크기와 입력 길이를 줄이는 방법도 함께 비교한다.

앱 배포를 준비할 때는 모델 · tokenizer의 사용 · 수정 · 재배포 조건과 실행 도구가 지원하는 Android 버전 · CPU 구조를 확인한다.

이해 확인과 완료 기준

☐ 모델 변환과 양자화가 서로 다른 작업임을 설명할 수 있다.

☐ 파일 크기와 실행 메모리를 구분해 측정한다.

☐ 품질 · 속도 · 메모리를 근거로 배포할 모델을 고를 수 있다.

산출물: POC C와 모델별 비교표.

자료: ONNX Runtime 모바일 배포, ONNX 모델 양자화.

과목 8. 생성형 요약과 효율적인 추가학습 입문

목표: 추출 요약과 생성형 요약을 비교하고, 생성형 모델의 적용 가능성과 추가학습 필요성을 판단한다.

쉬운 개념

- 생성형 요약: 원문의 핵심 내용을 새 문장으로 작성하는 방식이다.
- Instruct 모델: 지시를 따르도록 조정된 모델이다.
- 프롬프트: 모델에 작업과 조건을 전달하는 입력이다.
- 문맥 길이: 한 번의 처리에서 모델이 다룰 수 있는 토큰 범위다.
- 환각: 원문에 없는 내용을 만들어 내거나 사실을 바꾸는 오류다.

작은 예제

원문:

> 앱 업데이트는 금요일에 배포된다. 주요 변경은 PDF 요약 기능이다. 설정 화면의 아이콘도 변경된다.

출력 형식의 예시:

- 추출 요약: '앱 업데이트는 금요일에 배포된다. 주요 변경은 PDF 요약 기능이다.'
- 생성형 요약: '금요일 업데이트로 PDF 요약 기능이 추가된다.'

어느 방식이든 핵심을 놓칠 수 있다. 생성형 요약에서는 원문에 없는 배포 시간이나 기능을 덧붙이지 않았는지도 확인한다.

직접 실습

1. 기존 소형 다국어 생성 모델을 PC에서 실행한다.
2. '원문에 있는 내용만 사용해 한국어 세 문장 이내로 요약하라'는 요청부터 시작한다.
3. 출력 길이와 생성 설정을 바꾸고 결과를 비교한다.
4. 긴 문서는 문단별 요약 → 통합 요약을 실험하고 핵심 누락을 확인한다.
5. 같은 문서의 추출 요약과 생성형 요약을 비교한다.
6. LiteRT-LM 호환 모델로 Android CPU 실행을 실험한다. 학습 전에 모델 형식, 한국어 품질과 배포 가능성을 확인한다.

생성형 모델은 작은 분류기보다 실행 부담이 크다. 대상 기기의 메모리 · 실행 환경에서 검증하고, 실행이 어렵거나 품질이 부족하면 앱의 추출 요약을 유지한다. 생성형 모델 학습은 PC에서 계속할 수 있다.

중급 선택 실습: LoRA와 QLoRA

- LoRA: 모델의 일부 추가 가중치를 학습해 학습 비용을 줄이는 방법이다.
- QLoRA: 양자화한 기반 모델 위에서 LoRA를 학습해 학습 메모리를 줄이는 방법이다.

예를 들어 원하는 요약 형식이 계속 지켜지지 않는다면, 문서와 원하는 요약의 예제 쌍을 준비해 추가학습을 실험한다. 먼저 예제의 정답 품질과 기본 모델의 한계를 확인한다.

LoRA를 적용했다고 최종 배포 모델이 자동으로 작아지는 것은 아니다. 학습 결과의 변환 · 병합 · 배포 양자화는 별도로 확인한다. 처음에는 작은 학습 예제로 절차를 익히고, 원본보다 좋아졌는지 같은 평가 자료로 비교한다.

이해 확인과 완료 기준

☐ 생성형 요약에서 사실 오류와 핵심 누락을 찾아낼 수 있다.

☐ 프롬프트 수정, 추가학습, 모델 교체 중 무엇을 시도할지 이유를 설명할 수 있다.

☐ LoRA · QLoRA와 배포 양자화의 목적을 구분할 수 있다.

☐ 생성형 요약을 앱에 적용할 가치와 실행 가능성을 평가할 수 있다.

산출물: POC D와 추출 요약 · 생성형 요약 비교 기록. LoRA는 선택 실습이다.

자료: Hugging Face 요약 가이드, LiteRT-LM Android, PEFT 양자화와 QLoRA.

4. POC 범위와 검증 기준

단계별 결과물

POC	입력과 출력	핵심 검증
A: PC 문서 분석기	텍스트 → 주제 분류와 핵심 문장 최대 3개	저장·재로딩, 분류 평가, 요약 직접 검토
B: 첫 오프라인 앱	TXT·단순한 텍스트 PDF → 분류와 추출 요약	PC와 결과 일치, 오프라인, 취소·재실행
C: 추가학습·경량화 앱	같은 문서 → 신경망 분류와 기존 추출 요약	원본·추가학습·양자화 모델의 품질과 실행 비용
D: 생성형 요약 실험	같은 문서 → 생성형 요약	사실 일치, 핵심 포함, 기기 실행 가능성

첫 PDF 범위는 글자 읽기 순서가 명확한 단일 열 문서로 제한한다. 스캔, 복잡한 표 · 다단 문서, Word · 한글 파일은 별도 확장 과제로 둔다.

데이터와 품질

- 실습 데이터는 사용 권한이 있는 공개 자료나 직접 작성한 문서를 사용한다.
- 짧은 문서 30개로 시작해 분류 학습 단계에서 약 300개로 확장한다. 주제별 약 100개를 기본으로 한다.
- 학습 · 검증 · 테스트는 대략 70:15:15로 나누되 종류별 표본을 유지한다. 같은 원문의 조각, 중복 문서, 비슷한 문서 버전은 같은 쪽에 둔다.
- 학습 데이터로 모델과 전처리 기준을 학습하고, 검증 데이터로 설정을 고른다. 테스트 데이터는 최종 평가에 사용한다.
- 분류의 초기 학습 목표는 macro-F1 0.80이다. 미달하면 데이터 부족, 모호한 라벨, 입력 잘림과 오답 유형을 분석한다.
- 요약은 별도 문서 30개를 사람이 읽어 핵심 내용 포함, 문맥 유지, 중복과 사실 오류를 평가한다. 자동 점수 하나로 요약 품질을 판정하지 않는다.
- 양자화 품질 하락의 초기 허용치는 macro-F1 2%p 이내다. 이 수치는 학습용 비교 기준이며 제품 출시 기준은 실제 사용 문서로 다시 정한다.

앱 동작과 실행 비용

- [ ] 같은 문서 20개로 PC와 Android의 전처리 · 예측 결과를 비교한다.
- [ ] 모델과 필요한 파일을 준비한 뒤 비행기 모드에서 실행한다.
- [ ] 빈 문서, 텍스트가 없는 PDF, 읽을 수 없는 파일을 안내한다.
- [ ] 연속 실행과 취소 후 재실행을 확인한다.
- [ ] 모델 준비 · 로딩과 실제 분석 시간을 나누어 기록한다.
- [ ] 실기기에서 모델 크기, 메모리 사용량과 반복 실행 시간을 기록한다.

실행 도구와 모델 파일의 최초 다운로드가 필요한 경우가 있다. 첫 POC의 오프라인 기준은 **모델 준비 이후 문서 분석에 네트워크가 필요하지 않은 상태**다. Android 앱은 문서 내용을 분석 서버로 보내지 않는다.

5. 학습 기록과 다음 단계 판단

각 과목이 끝날 때 아래 네 가지를 남긴다.

1. **설명**: 핵심 개념을 자신의 말로 정리한 내용
2. **실습**: 다시 실행할 수 있는 코드나 앱
3. **측정**: 품질 · 속도 · 메모리 등 비교 결과
4. **판단**: 다음에 무엇을 바꿀지와 그 이유

처음에는 한 문서를 읽어 결과를 확인하는 수준으로 시작한다. 익숙해지면 입력을 바꾸고, 일부 설정을 바꾸고, 마지막으로 두 방법을 비교한다. 설명을 이해한 것과 직접 구현할 수 있는 것을 구분하며 진행한다.

기본 과정의 완료 기준은 **오프라인 POC, 분류 모델 추가학습, 양자화 비교를 직접 수행하고 선택 이유를 설명하는 것**이다. 생성형 요약과 LoRA는 그 위에서 확장한다. 대규모 사전학습, Transformer 전체 구현, 수식 증명, 지식 증류 · 가지치기 · 양자화 인지 학습의 본격적인 구현은 기본 과정 이후로 둔다.